Temario Arduino Nivel Licenciatura Ingeniería

https://gemini.google.com/app/ec8080c86598250f

User prompt: Dame un temario para impartir un curso de arduino nivel licenciatura

Response: Este programa semestral (16 semanas, 64 horas totales) aborda la plataforma Arduino desde una perspectiva rigurosa de ingeniería, trascendiendo el uso de librerías comerciales para enfocarse en arquitectura de microcontroladores, manejo directo de registros, acondicionamiento de señales y diseño de sistemas embebidos.

Sistemas Embebidos y Programación Aplicada con Arduino

Unidad 1: Arquitectura de Microcontroladores y Entorno Interno

- **1.1** Arquitectura Harvard vs. Von Neumann; análisis del microcontrolador ATmega328P / ATmega2560.
- **1.2** Organización de memoria: Flash, SRAM y EEPROM.
- **1.3** Ciclo de instrucción, reloj del sistema y distribución de potencia.
- **1.4** Programación a bajo nivel: manipulación de registros de E/S (`DDRx`, `PORTx`, `PINx`) mediante operadores a nivel de bits.

Unidad 2: Interrupciones, Temporización y PWM Hardware

- **2.1** Vectores de interrupción y procesamiento de interrupciones externas (`INT0`, `INT1`, `PCINT`).
- **2.2** Arquitectura de contadores/temporizadores internos (Timer0, Timer1, Timer2).
- **2.3** Modulación por Ancho de Pulso (PWM): Modos *Fast PWM* y *Phase Correct PWM*.
- **2.4** Generación de señales de precisión y manejo del perro guardián (*Watchdog Timer*).

Unidad 3: Conversión Analógica-Digital y Acondicionamiento

- **3.1** Funcionamiento del ADC de aproximaciones sucesivas (SAR), prescaler y voltaje de referencia.
- **3.2** Acondicionamiento analógico: amplificadores operacionales, filtrado pasivo/activo y aislamiento eléctrico.
- **3.3** Calibración, resolución, error de cuantificación y técnicas de sobremuestreo (*oversampling*).

Unidad 4: Protocolos de Comunicación Serie e Interfaz de Periféricos

- **4.1 UART/USART:** Configuración de baudrate, registros de transmisión/recepción y comunicación asíncrona.

- **4.2 SPI:** Arquitectura Maestro-Esclavo, modos de polaridad/fase de reloj e interfaz con memoria/sensores de alta velocidad.
- **4.3 I2C (TWI):** Protocolo de direccionamiento, líneas SDA/SCL, manejo de colisiones y sensores integrados (MPU6050, RTC).

Unidad 5: Eficiencia Energética, Sistemas RTOS y Almacenamiento

- **5.1** Modos de bajo consumo (*Sleep Modes*) y estrategias de administración de energía.
- **5.2** Lectura y escritura no volátil: EEPROM interna y módulo tarjeta SD vía SPI.
- **5.3** Introducción a Sistemas Operativos en Tiempo Real (FreeRTOS para AVR): Tareas, semáforos, colas y planificador.

Unidad 6: Integración e Internet de las Cosas (IoT)

- **6.1** Integración con plataformas de 32 bits (ESP32 / NodeMCU) usando el ecosistema Arduino Core.
- **6.2** Conectividad de red: Protocolos HTTP, WebSockets y MQTT para transmisión de telemetría.
- **6.3** Proyecto Integrador: Diseño y construcción de un nodo de instrumentación o control autónomo.

Evaluación Propuesta

- **Exámenes teóricos-prácticos:** 30%
- **Prácticas de laboratorio semanal:** 35%
- **Proyecto integrador final (hardware/firmware):** 35%

User prompt: Recomiéndame algunos libros para estos temas

Response: Para acompañar un temario de nivel ingeniería, es conveniente combinar **textos teóricos de arquitectura/interfaz**, **libros enfocados en el microcontrolador subyacente (AVR)** y **manuales de patrones de diseño/sistemas en tiempo real**.

A continuación, una selección bibliográfica organizada por su función académica dentro del curso:

1. Fundamentos y Arquitectura de Microcontroladores (Base del curso)

- ***The AVR Microcontroller and Embedded Systems: Using Assembly and C***
  - **Autores:** Muhammad Ali Mazidi, Sarmad Naimi, Sepehr Naimi.
  - **Por qué incluirlo:** Es el texto clásico por excelencia para entender la arquitectura interna del ATmega328P/ATmega2560. Muestra cómo funcionan los registros, la

memoria, los contadores/temporizadores, el ADC y los periféricos de comunicación (UART, SPI, I2C) utilizando C y Ensamblador. Es idóneo para las **Unidades 1, 2 y 4**.

- ***Embedded Controllers Using C and Arduino***
    - **Autor:** James M. Fiore.
    - **Por qué incluirlo:** Libro de texto abierto orientado a ingeniería que trasciende el uso de las librerías estándar de Arduino, enfocándose en el acceso directo a puertos (`PORTx`, `DDRx`), operaciones a nivel de bits y eficiencia de ejecución en C. Cubre muy bien la transición entre el ecosistema Arduino y la programación de bajo nivel.

2. Diseño de Software Embebido y Arquitectura (Nivel Avanzado)

- ***Making Embedded Systems: Design Patterns for Great Software***
    - **Autora:** Elecia White.
    - **Por qué incluirlo:** Esencial para que el estudiante aprenda a estructurar código profesional en sistemas con recursos limitados. Aborda patrones de diseño para drivers, gestión de energía, manejo de interrupciones, máquinas de estados finitos y optimización de memoria. Excelente apoyo para las **Unidades 2 y 5**.
- ***Real-Time Operating Systems for Small Microcontrollers***
    - **Autor:** Richard Barry (o documentación oficial de FreeRTOS).
    - **Por qué incluirlo:** Proporciona los conceptos teóricos y prácticos de la integración de un RTOS (planificación de tareas, colas, semáforos y exclusión mutua) en arquitecturas de 8 y 32 bits, cubriendo los objetivos de la **Unidad 5**.

3. Sistemas IoT y Microcontroladores de 32 Bits (Unidad de Cierre)

- ***Developing IoT Projects with ESP32***
    - **Autor:** Dogan Ibrahim.
    - **Por qué incluirlo:** Ideal para la **Unidad 6**, ya que aborda la transición del entorno AVR de 8 bits a la arquitectura de 32 bits basada en ESP32 utilizando el mismo core de programación de Arduino, integrando conectividad Wi-Fi, Bluetooth y protocolos de red como MQTT y HTTP.

4. Lecturas de Referencia Técnica Indispensables

- ***Atmel ATmega328P / ATmega2560 Datasheets*** **(Microchip Technology)**
    - En un curso de licenciatura, el manejo de la hoja de datos oficial es una competencia clave. Se recomienda incluir la *Datasheet* oficial como material de lectura obligatoria desde la primera semana para la consulta de mapas de registros, diagramas de tiempo y tablas de vectores de interrupción.

Sources

1. AVR Microcontroller & Embedded Systems: Assembly & C - E-Book
2. Embedded Controllers Using C and Arduino - 2e - Open Textbook Library
3. Making Embedded Systems: Design Patterns for Great Software - Goodreads
4. GitHub - eleciawhite/making-embedded-systems: A repository partner with Making Embedded Systems, 2nd edition

---

User prompt: ¿Qué libros me recomiendas si quiero orientar el curso hacia control digital, procesamiento de señales (DSP) e instrumentación con Arduino?

Response: Para orientar el curso de nivel licenciatura hacia **control digital**, **procesamiento digital de señales (DSP)** e **instrumentación**, la literatura debe combinar la teoría de sistemas dinámicos y muestreo con la implementación práctica en C/C++ sobre el hardware.

A continuación, se presentan los textos recomendados organizados por área temática:

1. Control Digital Integrado y Mapeo en Sistemas Embebidos

- ***Microcontroller Based Applied Digital Control***
  - **Autor:** Dogan Ibrahim.
  - **Por qué incluirlo:** Aborda el diseño de sistemas de control en tiempo real (PID, discreto, espacio de estados) e implementación en microcontroladores. Muestra cómo pasar de modelos teóricos en transformada Z a algoritmos programados en C.
- ***Digital Control Engineering: Analysis and Design***
  - **Autor:** M. Sami Fadali y Antonio Visioli.
  - **Por qué incluirlo:** Proporciona el soporte teórico de control en tiempo discreto: función de transferencia en Z, estabilidad, controladores PID discretos y representación en el espacio de estados. Es el libro base de teoría para que los alumnos justifiquen los algoritmos que subirán a la tarjeta.

2. Procesamiento Digital de Señales (DSP) Aplicado

- ***Digital Signal Processing Using Arm Cortex-M Based Microcontrollers***
  - **Autor:** Cem Ünsalan, M. Erkin Yücel, H. Deniz Gürhan.
  - **Por qué incluirlo:** Aunque enfocado en núcleos ARM (Cortex-M), es una referencia práctica para implementar conceptos de DSP. Muestra cómo realizar filtrado FIR e IIR, FFT y muestreo práctico mediante código en C de manera directa.
- ***The Scientist and Engineer's Guide to Digital Signal Processing***
  - **Autor:** Steven W. Smith (Disponible libremente en línea).
  - **Por qué incluirlo:** Texto clásico de DSP con enfoque intuitivo y orientado al cálculo eficiente. Explica conceptos clave como la Transformada Discreta de Fourier (DFT),

convolución, diseño de filtros y aliasing, facilitando su traducción a algoritmos dentro del código de la plataforma.

3. Instrumentación Electrónica y Acondicionamiento de Señales

- ***Elements of Electronic Instrumentation and Measurements***
  - **Autores:** Joseph J. Carr y David A. Bell.
  - **Por qué incluirlo:** Esencial para enseñar el aislamiento, la ampliación y el filtrado analógico antes de entrar al convertidor ADC. Cubre la teoría de puentes de medición, amplificadores de instrumentación y linealización de sensores.
- ***Sensors and Transducers: Characteristics, Applications, Instrumentation, Interfacing***
  - **Autor:** Ian R. Sinclair.
  - **Por qué incluirlo:** Enfocado en la interfaz física entre los transductores (temperatura, presión, celda de carga, encoders) y las entradas analógicas o digitales del microcontrolador.

Sugerencia para la arquitectura del laboratorio

Para las prácticas de **DSP e Instrumentación**, la tarjeta Arduino Uno tradicional (ATmega328P de 8 bits) puede quedarse limitada por velocidad de reloj y resolución del ADC (10 bits). Se recomienda utilizar una placa con núcleo ARM o ESP32 (como la familia Arduino Due o placas SAMD21/ESP32) para las unidades de DSP y control, manteniendo la compatibilidad con el ecosistema de desarrollo de Arduino.

Sources

1. CONTROL TECHNIQUE WITH MICROCONTROLLERS - European Commission
2. Digital-Signal-Processing-using-Arm-Cortex-M-based-Microcontrollers - GitHub
3. DSP Education Using Arduino Platform | PDF | Digital Signal Processing - Scribd
4. Essentials of Signal Conditioning Techniques | PDF | Analog To Digital Converter - Scribd

---

User prompt: Diseña una lista de 5 prácticas de laboratorio enfocadas en DSP y Control Digital usando Arduino.

Response: A continuación se presenta una propuesta de **5 prácticas de laboratorio** estructuradas progresivamente para nivel licenciatura, enfocadas en la implementación práctica de **Control Digital** y **Procesamiento Digital de Señales (DSP)** utilizando la plataforma Arduino.

Práctica 1: Adquisición de Señales, Muestreo y Análisis de Aliasing

- **Objetivo:** Comprender el impacto del periodo de muestreo (T_s), la frecuencia de Nyquist y el fenómeno de *aliasing* en el dominio discreto.
- **Hardware requerido:** Arduino (Uno/Due/ESP32), generador de funciones, osciloscopio y potenciómetro.

- **Desarrollo:**

    1. Configurar un temporizador por hardware (*Timer Interrupt*) para muestrear un canal analógico del ADC a una frecuencia fija conocida (ej. $f_s = 100\ \text{Hz}$).
    2. Inyectar mediante un generador de funciones una señal senoidal de frecuencia creciente ($10\ \text{Hz}$, $40\ \text{Hz}$, $60\ \text{Hz}$, $120\ \text{Hz}$).
    3. Reconstruir la señal enviando los datos por el puerto serie (*Serial Plotter*) para visualizar el aliasing cuando $f_{in} > \frac{f_s}{2}$.
    4. Modificar el temporizador para ajustar dinámicamente la frecuencia de muestreo y calcular el error de cuantificación.

Práctica 2: Diseño e Implementación de Filtros Digitales IIR y FIR

- **Objetivo:** Diseñar e implementar filtros digitales en tiempo real para la eliminación de ruido de alta frecuencia en señales de sensores.
- **Hardware requerido:** Arduino, sensor de temperatura/LDR, generador de funciones/señal con ruido, osciloscopio.
- **Desarrollo:**

    1. Diseñar teóricamente un filtro pasa-bajas IIR de primer orden (promedio exponencial ponderado) y un filtro FIR de media móvil (*Moving Average*) de N muestras.
    2. Obtener la ecuación en diferencias de cada filtro:
        - **IIR:** $y[n] = \alpha \cdot x[n] + (1 - \alpha) \cdot y[n-1]$
        - **FIR:** $y[n] = \frac{1}{N} \sum_{k=0}^{N-1} x[n-k]$
    3. Programar las ecuaciones dentro de la rutina de interrupción (ISR) asegurando tiempos de ejecución eficientes mediante operaciones con punto fijo o coma flotante básica.
    4. Comparar en tiempo real la respuesta en frecuencia, latencia y atenuación de ruido de ambos filtros frente a una señal contaminada.

Práctica 3: Discretización e Implementación de un Controlador PID Digital

- **Objetivo:** Discretizar un algoritmo PID analógico e implementarlo en un sistema de control de posición o velocidad de un motor DC con encoder.
- **Hardware requerido:** Arduino, motor DC con encoder cuadratura, driver puente H (L298N o similar), fuente de alimentación externa.
- **Desarrollo:**

    1. Obtener la representación discreta del PID utilizando el método de Euler hacia atrás o Tustin (Bilineal):

$$u[n] = K_p e[n] + K_i T_s \sum e[n] + \frac{K_d}{T_s}(e[n] - e[n-1])$$

2. Configurar la lectura del encoder mediante interrupciones externas (`ATTACH_INTERRUPT`) para calcular la posición/velocidad real.
3. Implementar el algoritmo PID dentro de un bucle de tiempo estricto (T_s constante) incluyendo técnicas antibloqueo del término integral (*Anti-windup*).
4. Ajustar las ganancias (K_p, K_i, K_d) y analizar la respuesta al escalón del sistema.

Práctica 4: Control por Realimentación de Estados y Asignación de Polos en Tiempo Discreto

- **Objetivo:** Diseñar e implementar un controlador en el espacio de estados discreto para un sistema de segundo orden (ej. sistema térmico o levitador magnético/péndulo).
- **Hardware requerido:** Arduino (preferente de 32 bits como Due o ESP32), planta física de segundo orden (o circuito RC en cascada como emulador analógico).
- **Desarrollo:**
    1. Obtener la matriz del sistema en tiempo continuo $(\mathbf{A}, \mathbf{B}, \mathbf{C}, \mathbf{D})$ y discretizarla para obtener $(\mathbf{A}_d, \mathbf{B}_d)$ dado un T_s.
    2. Calcular la matriz de ganancias de realimentación $\mathbf{K}$ mediante la fórmula de Ackermann para ubicar los polos en bucle cerrado dentro del círculo unitario.
    3. Programar la ley de control vectorial $u[n] = -\mathbf{K}\mathbf{x}[n] + k_r r[n]$ utilizando operaciones matriciales optimizadas.
    4. Validar la estabilidad, tiempo de asentamiento y sobretiro del sistema frente a perturbaciones.

Práctica 5: Espectrograma y Análisis Frecuencial en Tiempo Real (FFT)

- **Objetivo:** Aplicar la Transformada Rápida de Fourier (FFT) sobre señales de audio o vibración para la detección de frecuencias características.
- **Hardware requerido:** Arduino (ESP32 / SAMD21 / Due recomendados por la carga de cómputo), módulo micrófono electret/MAX4466 o acelerómetro MEMS (MPU6050).
- **Desarrollo:**
    1. Capturar un bloque o ventana de datos de $N = 64$ o 128 muestras aplicando una función de ventana (*Hann* o *Hamming*) para reducir el furgón espectral (*leakage*).
    2. Ejecutar la librería de FFT (ej. `arduinoFFT`) sobre la ventana de datos capturada.

3. Extraer la magnitud de los componentes frecuenciales y detectar la frecuencia fundamental de la señal de entrada.
4. Mostrar el espectro de frecuencias en un display OLED o enviarlo vía serial hacia una interfaz gráfica en MATLAB/Python.

Estructura de Entregables por Práctica

Para asegurar el rigor a nivel licenciatura, cada reporte de laboratorio debe incluir:

- **Modelado Teórico:** Derivación matemática de las ecuaciones en diferencias o matrices discretas.
- **Código Fuente:** Implementación comentada con énfasis en el tiempo de procesamiento dentro de la ISR.
- **Análisis de Desempeño:** Capturas de osciloscopio o gráficas comparativas entre la simulación teórica (MATLAB/Simulink) y la respuesta real del hardware.
